Ex 1.1

(a) a := [7, 5, 4, 9, 6, 2, 4]

insertion Sort steps:

7	5	4	9	6	2	4

↑ sorted

step 1:

5	7	4	9	6	2	4

step 2:

4	5	7	9	6	2	4

step 3:

4	5	7	9	6	2	4

step 4:

4	5	6	7	9	2	4

step 5:

2	4	5	6	7	9	4

step 6:

2	4	4	5	6	7	9

Ex 1.1

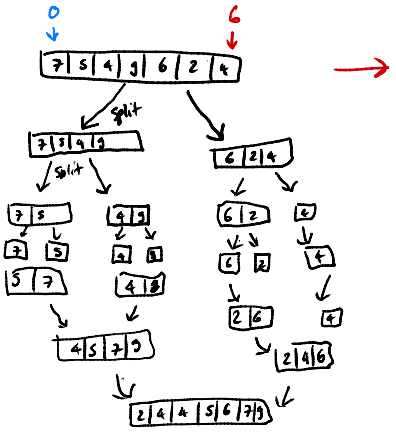
(b) p = low
q = mid
r = high

5	7	8	1	4	6

Temp := int[5 − 0]

step 0:	a[i] = 5 ;	a[j] = 1	tmp[0] = 1	i ⇒ 0	j ⇒ 4
step 1:	a[i] = 5 ;	a[j] = 4	tmp[1] = 4	i ⇒ 0	j ⇒ 5
step 2:	a[i] = 5 ;	a[j] = 6	tmp[2] = 5	i ⇒ 1	j ⇒ 5
step 3:	a[i] = 7 ;	a[j] = 6	tmp[3] = 6	i ⇒ 1	j ⇒ 6
step 4:			tmp[4] = 7	i ⇒ 2	j ⇒ 6
step 5:			tmp[5] = 9	i ⇒ 3	j ⇒ 6

(c)



→

sort_aux(0,6)
↳ sort_aux(0,3)
↳ sort_aux(0,1)
↳ sort_aux(0,0)
↳ sort_aux(1,1)
↳ sort_aux(2,3)
↳ sort_aux(2,2)
↳ sort_aux(3,3)
↳ sort_aux(4,6)
↳ sort_aux(4,5)
↳ sort_aux(4,4)
↳ sort_aux(5,5)
↳ sort_aux(6,6)

base calls = 15 recusive calls = 15 − 1

nuber of leafs n = 15 ; splits = 14

2 · 15 − 1 = 29 ⇒ nubers of calls
→ in a Binary Tree

Merge sort performs this pattern:

→ split by indices
→ recurse on left part
→ recurse on right part
→ stop when subarray size is 1

it depends only on the input size!

it doesnt also matter if its already sorted

⇒ calls = 2n − 1 ⇒ T(15) = 29